

# Reviewer Pack — Hodge Laplacian $b_1$ of Edge-Triangle Star-Complex Bounds SoS...

Entry ea9308fd8a1c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 16:05:44 UTC

## Hodge Laplacian $b_1$ of Edge-Triangle Star-Complex Bounds SoS Degree of Tseitin

Entry ID: ea9308fd8a1c **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 16:05:44 UTC

### 1. Conjecture

**Field A** (mathematical branch): Discrete combinatorial Hodge theory of finite simplicial 2-complexes (Eckmann 1944; Friedman ‘Computing Betti Numbers via Combinatorial Laplacians’; Jiang-Lim-Yao 2011):  $h_1(\Sigma) = \text{nullity of the up-down Hodge Laplacian } L_1 = \partial_1^T \partial_1 + \partial_2 \partial_2^T \text{ over } \mathbb{Q}, \text{ equal to } \dim H_1(\Sigma; \mathbb{Q}) \text{ by Hodge’s theorem. Distinct from spectral graph theory (which uses only } L_0), \text{ from Newton-polytope/lattice methods, and from zeta-function counts; rarely deployed against SoS proof complexity.}$  **Field B** (complexity object): Sum-of-Squares (Lasserre/Parrilo) refutation degree  $d_{\text{SoS}}(T(G, c))$  of unsatisfiable Tseitin instances on graphs  $G$  with odd total  $F_2$  charge, accessed via the Schoenebeck/Grigoriev combinatorial proxy  $g^*(F) = \min\{|T| : \bigoplus\{C \in T\} a_C = 0 \in F_2^{V(G)} \text{ and } \bigoplus\{C \in T\} b_C = 1 \in F_2\}$  — the canonical SoS refutation-degree target for binary CSPs (Schoenebeck 2008; Grigoriev 2001).

#### Statement:

For a connected graph  $G$  with maximum degree  $\Delta$  and odd-charge assignment  $c$ , build the simplicial 2-complex  $\Sigma(G)$  whose 0-cells are the edges of  $G$ , whose 1-cells are pairs of  $G$ -edges sharing a vertex, and whose 2-cells are the triples of  $G$ -edges meeting at a common vertex (the local ‘vertex stars’). Let  $h_1(G) := \text{nullity}(L_1)$  over  $\mathbb{Q}$ . Conjecture:  $g^*(T(G, c)) \geq \lceil h_1(G) / (\Delta \cdot (\Delta - 1)) \rceil + 1$  for every unsatisfiable Tseitin instance  $T(G, c)$  with  $\Delta \leq 4$ , and this bound is strict for at least one instance per girth-class of  $G$ .

#### Rationale (proposer’s reasoning):

Schoenebeck-style SoS lower bounds for parity CSPs are governed by an  $F_2$  dependency-girth that is sensitive to  $F_2$  cancellation, while harmonic 1-cycles of the rationally lifted star complex  $\Sigma(G)$  detect independent topological ‘holes’ of the incidence structure that no single vertex-star 2-cell can fill — and these holes appear to obstruct precisely the local cancellation patterns short SoS pseudo-distributions exploit. Discrete Hodge theory is widely used in graph spectra and TDA but, to our knowledge, has not been deployed as a direct combinatorial obstruction inside the Lasserre/Parrilo hierarchy. If true, the conjecture yields a polynomial-time, basis-independent topological certificate of SoS hardness complementary to expansion-based bounds.

**Taxonomy category:** SOS\_HIERARCHY (status at proposal time: alive)

## 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 5a8e31d7a5640f37

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

Across 250 trials (50 random connected 3-regular graphs  $G$  for each  $n$  in  $\{8,10,12,14,16\}$  with random odd-parity charge  $c$ ), the conjecture is SUPPORTED iff at least 95% of trials satisfy  $g(T(G,c)) \geq \text{ceil}(h_1(G)/(\Delta(\Delta-1))) + 1$ , and FALSIFIED if any single trial yields  $g^*(T(G,c))$  strictly less than that bound.

## 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.93       | SAFE             | SAFE             |
| NATURAL PROOFS | SAFE    | 0.92       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.85       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.92       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - Tseitin sum-of-squares refutation degree simplicial complex Hodge Laplacian - combinatorial Hodge Laplacian Betti number Lasserre proof complexity Tseitin-Schoenebeck Grigoriev Tseitin SoS degree edge-triangle complex Laplacian nullity

**Top relevant hits considered:** - [http://arxiv.org/abs/2602.03763v1] Optimizing Weighted Hodge Laplacian Flows on Simplicial Complexes - [http://arxiv.org/abs/1807.05044v5] Random Walks on Simplicial Complexes and the normalized Hodge 1-Laplacian - [http://arxiv.org/abs/2205.02134v1] Hodge Decomposition and General Laplacian Solvers for Embedded Simplicial Complexes - [http://arxiv.org/abs/1908.06789v2] Combinatorial Proof of the Minimal Excludant Theorem - [http://arxiv.org/abs/2204.12218v2] Combinatorial and Hodge Laplacians: Similarity and Difference - [http://arxiv.org/abs/1903.05430v2] The construction problem for Hodge numbers modulo an integer - [http://arxiv.org/abs/2507.01005v1] Discovery and study of the eclipsing variable star Grigoriev 1 - [http://arxiv.org/abs/2310.12784v1] The nullity of the net Laplacian matrix of a signed graph

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=180.1s

### 5.1 Generated Python source

```
import sys
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
```

```

    max_row = i
    for j in range(i+1, n):
        if abs(A[j][i]) > abs(A[max_row][i]):
            max_row = j
    A[i], A[max_row] = A[max_row], A[i]
    for j in range(i+1, n):
        factor = A[j][i] / A[i][i]
        for k in range(n):
            A[j][k] -= factor * A[i][k]
    return A

def matrix_rank(A, tol=1e-9):
    A = gaussian_elimination(A)
    rank = 0
    for row in A:
        if any(abs(x) > tol for x in row):
            rank += 1
    return rank

def build_star_complex(G):
    n = len(G)
    edges = []
    edge_pairs = []
    vertex_stars = []

    for i in range(n):
        for j in range(i+1, n):
            if G[i][j] == 1:
                edges.append((i, j))
                for k in range(j+1, n):
                    if G[j][k] == 1 and G[k][i] == 1:
                        edge_pairs.append(((i, j), (j, k)))
                        vertex_stars.append(((i, j), (j, k), (k, i)))

    return edges, edge_pairs, vertex_stars

def compute_h1(G):
    n = len(G)
    edges, _, _ = build_star_complex(G)
    A1 = [[0] * n for _ in range(n)]
    for u, v in edges:
        A1[u][v] = 1
        A1[v][u] = 1

    return matrix_rank(A1)

def meet_in_the_middle(clauses):
    m = len(clauses)
    half = m // 2
    xor_dict = {}

    for subset in range(1 << half):
        a_C, b_C = 0, 0
        for j in range(half):
            if subset & (1 << j):
                a_C ^= clauses[j][0]

```

```

        b_C ^= clauses[j][1]
        xor_dict[(a_C, b_C)] = subset

    min_size = float('inf')
    for i in range(1 << half):
        a_T1, b_T1 = 0, 0
        for j in range(half):
            if i & (1 << j):
                a_T1 ^= clauses[j][0]
                b_T1 ^= clauses[j][1]

        a_T2, b_T2 = 0, 0
        for j in range(half, m):
            if i & (1 << (j - half)):
                a_T2 ^= clauses[j][0]
                b_T2 ^= clauses[j][1]

        xor_key = (a_T1 ^ a_T2, b_T1 ^ b_T2)
        if xor_key in xor_dict:
            size = bin(i).count('1') + bin(xor_dict[xor_key]).count('1')
            min_size = min(min_size, size)

    return min_size

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([8, 10, 12, 14, 16])
    G = [[0] * n for _ in range(n)]
    degree_sum = 0
    while True:
        for i in range(n):
            for j in range(i+1, n):
                if random.random() < 3 / (n - 1):
                    G[i][j] = G[j][i] = 1
                    degree_sum += 2

    if degree_sum % 2 != 0:
        c = [random.choice([0, 1]) for _ in range(n)]
    else:
        return {
            "metric_name": "g*(T(G,c))",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    h1 = compute_h1(G)
    clauses = []
    for i in range(n):
        for j in range(i+1, n):
            if G[i][j] == 1:
                for k in range(j+1, n):
                    if G[j][k] == 1 and G[k][i] == 1:
                        clauses.append(((i, j), (j, k)))

```

```

g_star = meet_in_the_middle(clauses)
bound = math.ceil(h1 / (4 * 3)) + 1

return {
    "metric_name": "g*(T(G,c))",
    "metric_value": g_star,
    "instances_tested": 1,
    "conjecture_holds": g_star >= bound,
    "counterexample": "" if g_star >= bound else f"g*={g_star} < {bound}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_trials = sum(r["instances_tested"] for r in results)
    mean_value = sum(r["metric_value"] for r in results) / total_trials
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / total_trials)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 180s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out after 180s without producing any RESULT line, so the pre-registered support condition could not be evaluated. Per Rule 2, a timeout yields INCONCLUSIVE. |

next: Reduce trial count or graph sizes (e.g.,  $n \leq 12$ ) and optimize the meet-in-the-middle SoS-degree search, or precompute  $h_1$  separately, to fit within the time budget.

## 11. Audit log (LLM calls)

**Total LLM calls:** 8

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 23372        |
| 2 | propose         | claude_max    | opus             | 0         | 0   | 200598       |
| 3 | preregistration | claude_max    | opus             | 0         | 0   | 14483        |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 4268         |
| 5 | novelty         | claude_max    | opus             | 0         | 0   | 9260         |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20570        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15987        |
| 8 | judge           | claude_max    | opus             | 0         | 0   | 6538         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 295074 ms total latency. Provider mix: {'claude\_max': 6, 'ollama\_remote': 2}

(full prompt+response transcripts available in `research/audit/ea9308fd8a1c.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ea9308fd8a1c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/ea9308fd8a1c.tar.gz` (if generated)